

BEE 4750 Lab 2: Monte Carlo

Name: Brune Boukobza

ID: bwb78

Due Date

Thursday, 10/02/25, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
Activating project at `~/Desktop/BEE 4750/lab-2-bwb78`
  Installed Libtiff_jll          v4.7.2+0
  Installed Xorg_libXfixes_jll   v6.0.2+0
  Installed Xorg_xcb_util_cursor_jll v0.1.6+0
  Installed Libmount_jll        v2.41.2+0
  Installed Libuuid_jll         v2.41.2+0
  Installed Plots                v1.41.1
Precompiling project...
 513.8 ms  Libuuid_jll
 520.3 ms  Libmount_jll
 771.6 ms  Libtiff_jll
 905.4 ms  QuadGK
 396.0 ms  Xorg_libXfixes_jll
 360.4 ms  Xorg_xcb_util_cursor_jll
```

```

724.5 ms    ColorVectorSpace → SpecialFunctionsExt
781.6 ms    Fontconfig_jll
801.7 ms    Cairo_jll
1089.3 ms   Qt6Base_jll
3026.1 ms   ColorSchemes
804.5 ms    HarfBuzz_jll
1114.3 ms   Qt6ShaderTools_jll
759.1 ms    libass_jll
778.4 ms    Pango_jll
3825.5 ms   Distributions
1122.5 ms   FFMPEG_jll
1396.4 ms   Qt6Declarative_jll
1021.2 ms   Distributions → DistributionsTestExt
1361.5 ms   FFMPEG
1947.2 ms   GR_jll
5667.5 ms   PlotUtils
2671.3 ms   GR
1673.0 ms   PlotThemes
2260.7 ms   RecipesPipeline
32747.4 ms   Plots
26 dependencies successfully precompiled in 45 seconds. 165 already precompiled.

```

```

using Random # random number generation
using Distributions # probability distributions and interface
using Statistics # basic statistical functions, including mean
using Plots # plotting

```

Overview

In this lab, we will conduct a Monte Carlo experiment and explore how sensitive Monte Carlo estimates are to the underlying probability distribution(s).

You should always start any computing with random numbers by setting a “seed,” which controls the sequence of numbers which are generated (since these are not *really* random, just “pseudorandom”). In Julia, we do this with the `Random.seed!()` function.

```
Random.seed!(1)
```

```
TaskLocalRNG()
```

It doesn't matter what seed you set, though different seeds might result in slightly different values. But setting a seed means every time your notebook is run, the answer will be the same.

Seeds and Reproducing Solutions

If you don't re-run your code in the same order or if you re-run the same cell repeatedly, you will not get the same solution. If you're working on a specific problem, you might want to re-use `Random.seed()` near any block of code you want to re-evaluate repeatedly.

Probability Distributions and Julia

Julia provides a common interface for probability distributions with the [Distributions.jl package](#). The basic workflow for sampling from a distribution is:

1. Set up the distribution. The specific syntax depends on the distribution and what parameters are required, but the general call is the similar. For a normal distribution or a uniform distribution, the syntax is

```
# you don't have to name this "normal_distribution"
#   is the mean and   is the standard deviation
normal_distribution = Normal( , )
# a is the upper bound and b is the lower bound; these can be set to +Inf or -Inf for an
uniform_distribution = Uniform(a, b)
```

There are lots of both [univariate](#) and [multivariate](#) distributions, as well as the ability to create your own, but we won't do anything too exotic here.

2. Draw samples. This uses the `rand()` command (which, when used without a distribution, just samples uniformly from the interval $[0, 1]$.) For example, to sample from our normal distribution above:

```
# draw n samples
rand(normal_distribution, n)
```

Putting this together, let's say that we wanted to simulate 100 six-sided dice rolls. We could use a [Discrete Uniform distribution](#).

```
dice_dist = DiscreteUniform(1, 6) # can generate any integer between 1 and 6
dice_rolls = rand(dice_dist, 100) # simulate rolls
```

100-element Vector{Int64}:

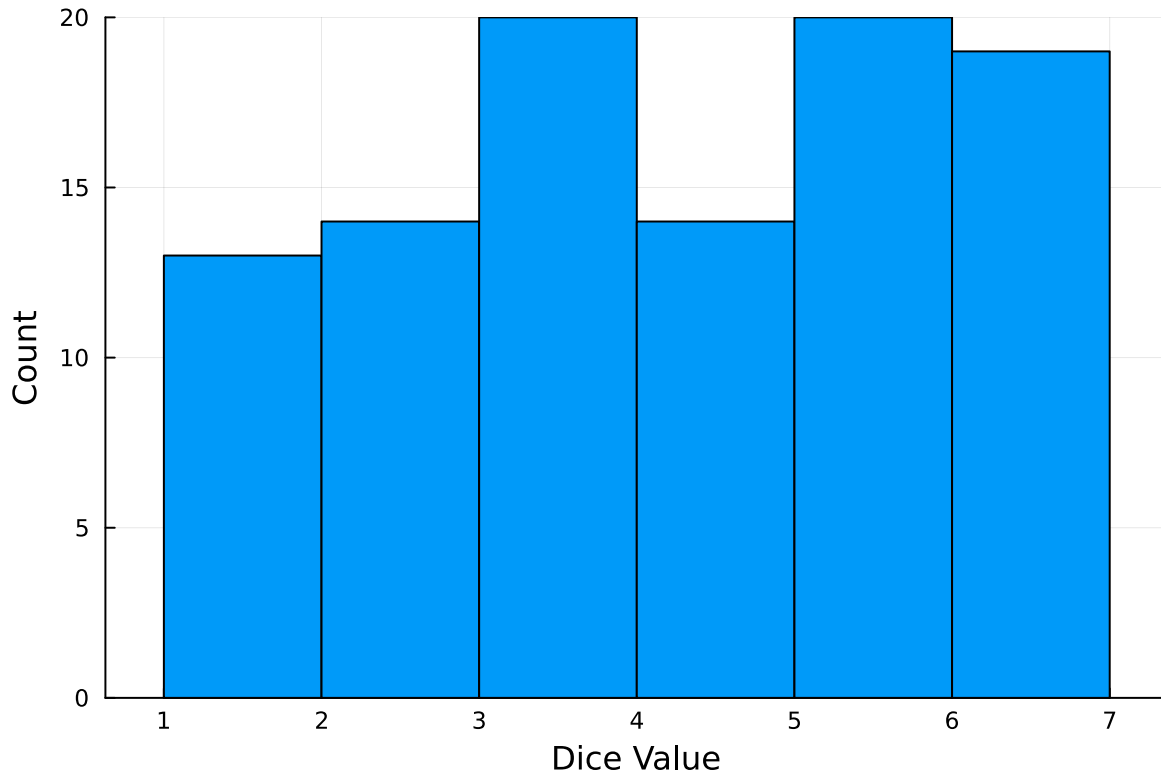
1

3
5
4
6
2
5
5
5
2

3
6
5
5
6
3
6
6
6

And then we can plot a histogram of these rolls:

```
histogram(dice_rolls, legend=:false, bins=6)  
ylabel!("Count")  
xlabel!("Dice Value")
```



Instructions

Remember to:

- Evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

Problem

A common engineering problem is to quantify flood risk, which is typically computed by propagating a flood hazard distribution through a *depth-damage function* relating flood depths to economic damages. A reasonable depth-damage function for a house without a basement is a bounded logistic function,

$$d(h) = \mathbb{1}_{h>0} \frac{L}{1 + \exp(-k(h - h_0))},$$

where d is the damage as a percent of total structure value, h is the water depth (relative to the house's ground floor elevation) in m, $\mathbb{1}_{x>0}$ is the indicator function, L is the maximum loss in USD, k is the slope of the depth-damage relationship, and h_0 is the inflection point. We'll assume $L = \$200,000$, $k = 0.8$, and $h_0 = 3$.

For this problem, suppose that we have two different probability distributions characterizing annual maxima flood depths:

1. $h \sim \text{LogNormal}(1.2, 0.3)$;
2. $h \sim \text{GeneralizedExtremeValue}(3, 0.9, -0.15)$.

Plot histograms of samples from these distributions and conduct a Monte Carlo experiment for annual maximum flood damages using each. Answer the following questions:

- What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions?
- How did you decide on your sample size?
- Why do you think the estimates differed or did not differ (you can also plot the depth-damage function to compare with the samples)?
- How might you decide (based on getting additional information, if needed) which distribution is “better”?

```
#plotting 4 histograms of samples from the distributions given

#Values given in problem
L = 200000      #max loss
k = 0.8         #slope
h0 = 3         #inflection point
N = 200000     #Monte Carlo sample size
rng = MersenneTwister(42) #random number generator

#Depth-damage function given in problem
damage(h) = h > 0 ? L / (1 + exp(-k * (h - h0))) : 0.0

#Distribution given in problem
dist_ln = LogNormal(1.2, 0.3)
dist_gev = GeneralizedExtremeValue(3, 0.9, -0.15)

#Samples
h_ln = rand(rng, dist_ln, N) #array of flood depth samples for ln
h_gev = rand(rng, dist_gev, N) #array of flood depth samples for gev

d_ln = damage.(h_ln) #applying damage function
```

```

d_gev = damage.(h_gev)

#Monte Carlo
println("LogNormal mean damage = \$", round(mean(d_ln), digits=2))
println("LogNormal 99% quantile = \$", round(quantile(d_ln, 0.99), digits=2))
println("GEV mean damage = \$", round(mean(d_gev), digits=2))
println("GEV 99% quantile = \$", round(quantile(d_gev, 0.99), digits=2))

##Plots
#Histogram of LogNormal depths
p1 = histogram(h_ln, bins=200, xlim=(0,12), xlabel="Depth h (m)", ylabel="Count",
               title="Histogram for LogNormal")
display(p1)

#Histogram of GEV depths
p2 = histogram(h_gev, bins=200, xlabel="Depth h (m)", ylabel="Count",
               title="Histogram for GEV")
display(p2)

#Histogram of damages for Log Normal
p3 = histogram(d_ln, bins=200, xlim=(0,L+1000), xlabel="Damage (USD)", ylabel="Count",
               title="Damage histogram for LogNormal")
display(p3)

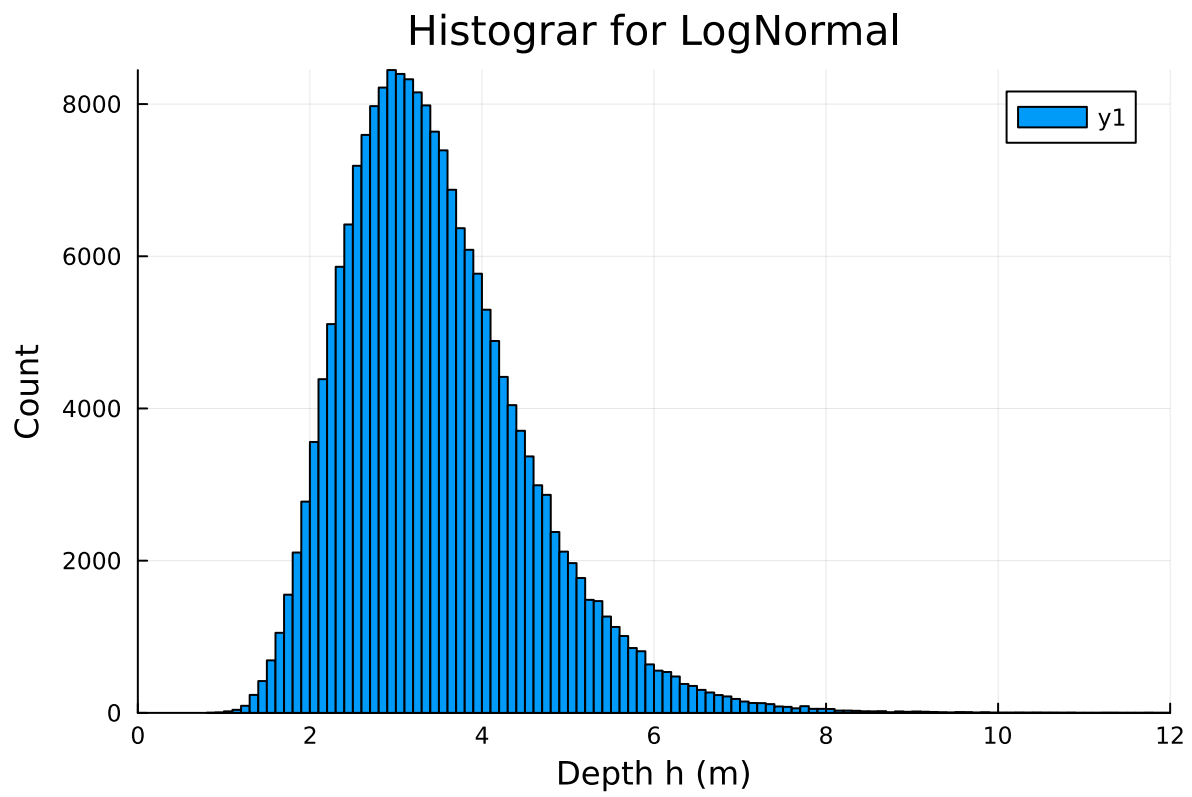
#Histogram of damages for GEV
p4 = histogram(d_gev, bins=50, xlim=(0,L+1000), xlabel="Damage (USD)", ylabel="Count",
               title="Damage histogram for GEV")
display(p4)

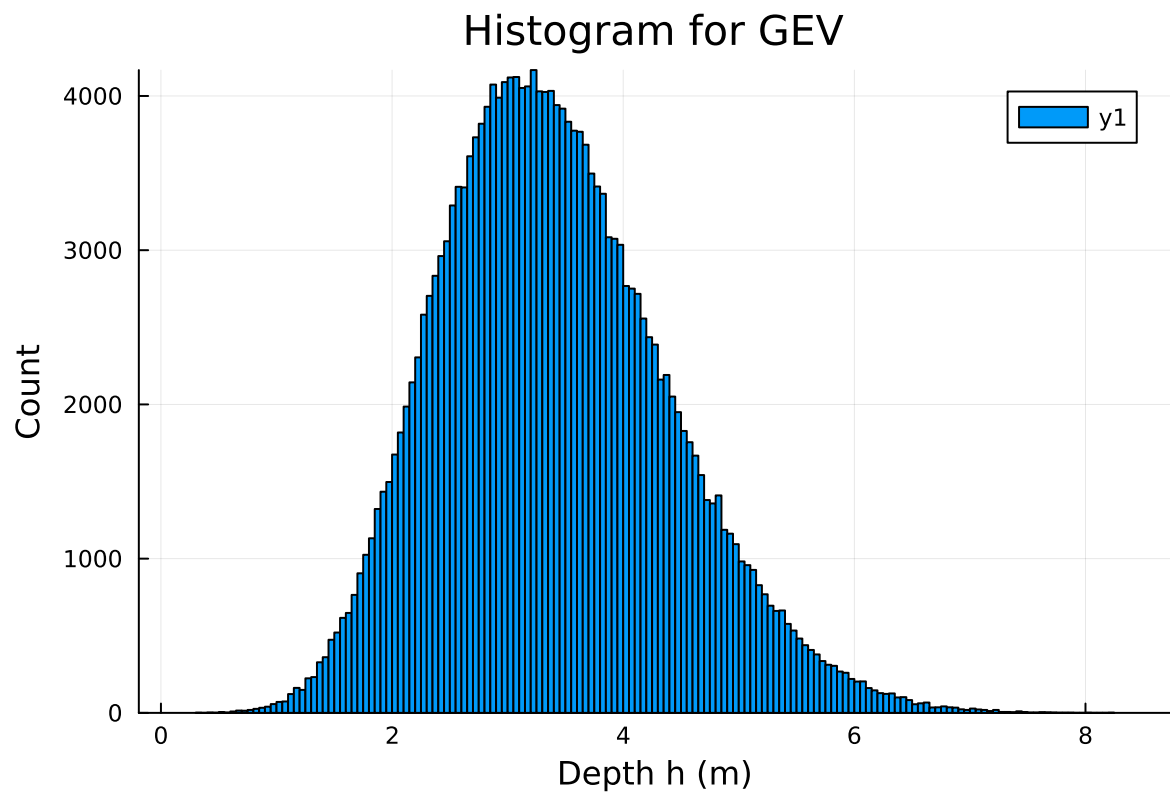
```

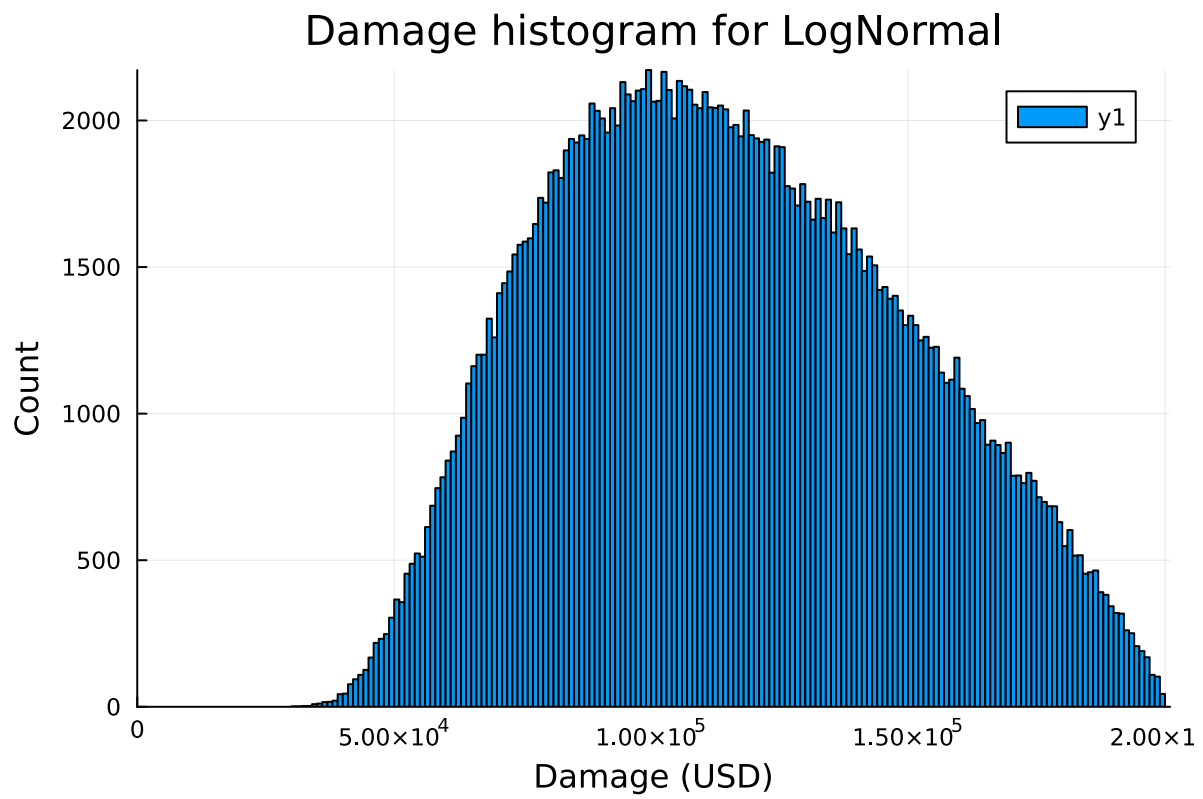
```

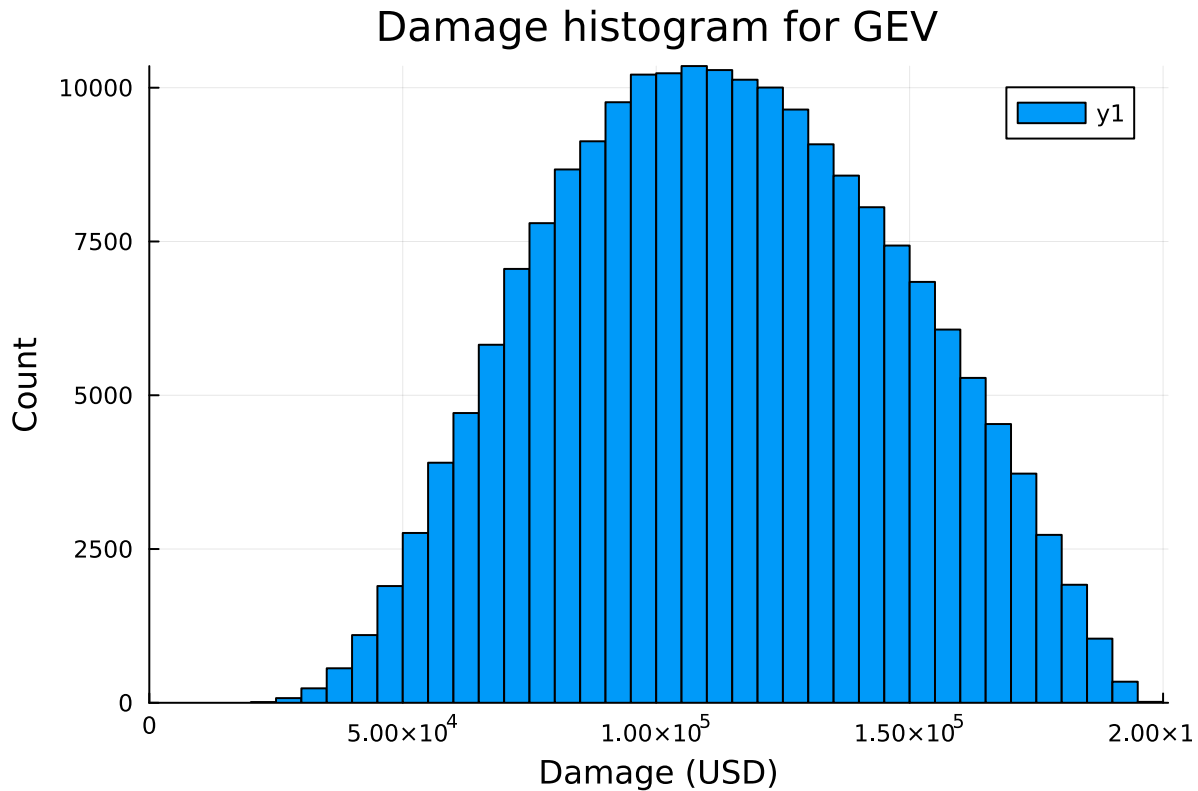
LogNormal mean damage = $115069.48
LogNormal 99% quantile = $189920.06
GEV mean damage = $113512.56
GEV 99% quantile = $183162.57

```









#Answer to questions

LogNormal mean damage = \$115069.48 LogNormal 99% quantile = \$189920.06 GEV mean damage = \$79825.88 GEV 99% quantile = \$200000.0

I chose the sample size of 200,000 because we are in a 99% quantile. As a result, we take the typical 1000 to reduce the monte carlo noise and divide it by 0.01 which results in 100,000. Then I multiplied by 2 to make sure the mean and tails are stable. There are fairly similar depth samples in the range that depth-damage curve changes the most. Differences in the extreme tails are due to the result that one distribution may produce many more samples above the inflection point. Additionally, the depth damage is bounded by 200000 dollars, which reduces the influence of extremely large depths. There are many ways to determine which distribution is best based on a real scenario, goodness of fit tests, and incorporating more data.

References

Put any consulted sources here, including classmates you worked with/who helped you.